

Class07: Lab

Joshua Robinson (PID: A18574862)

Table of contents

Background	1
K-means clustering	1
Hierarchical Clustering	7
Principal Component Analysis (PCA)	9
Analysis of UK food data	9
Data Import	9
Preview the first 6 rows	10
Note how the minus indexing works	11
Tidy Data	11
Exporatory Analysis	12
Barplots	12
PCA to the rescue	18
PCA of RNA-seq data	24

Background

Today we will explore some core machine learning methods that are very popular in bioinformatics. These include **clustering** and **dimensionality reduction**.

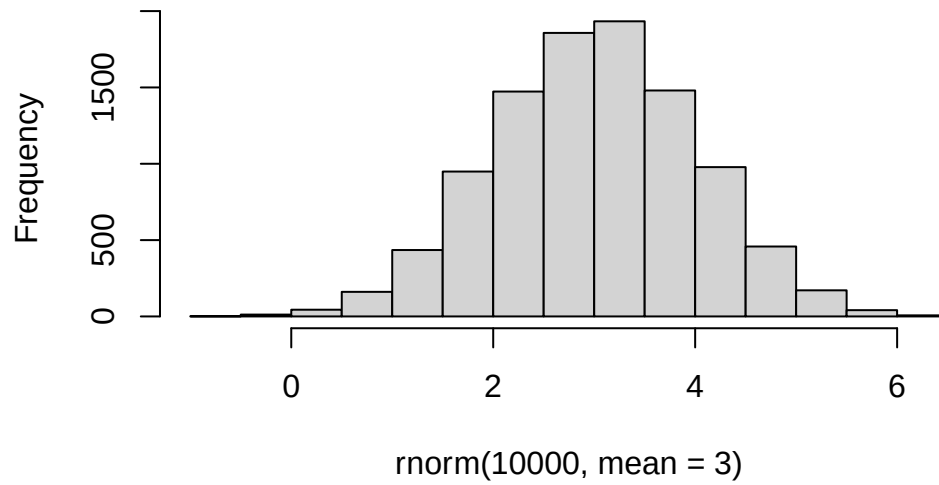
K-means clustering

The main function in “base” R for K-means clustering is called `kmeans()`

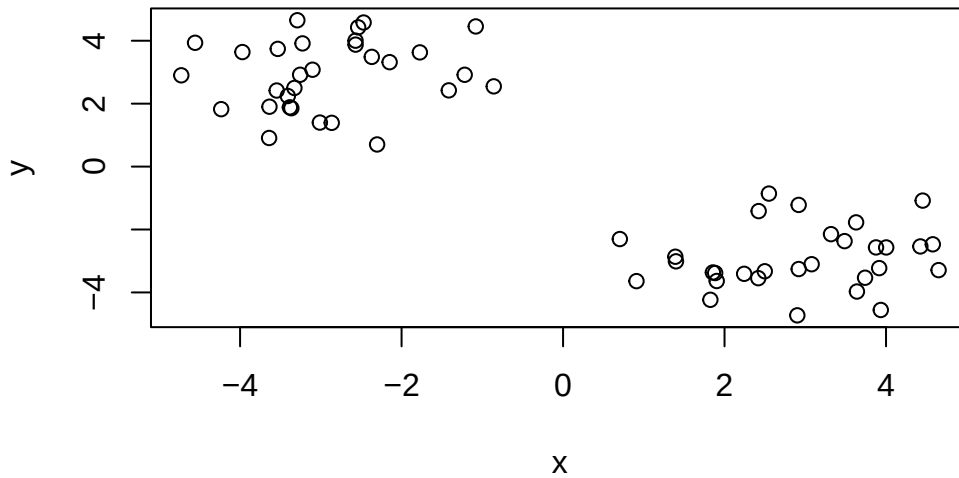
Before we go too deep let’s make up some “simple” data that we can cluster and know if we are getting a good answer or not. To do this we can use the `rnorm()` function:

```
hist(rnorm(10000, mean=3))
```

Histogram of rnorm(10000, mean = 3)



```
x <- c(rnorm(30, -3), rnorm(30, +3))  
z <- cbind(x=x, y=rev(x))  
plot(z)
```



Now we can run `kmeans()` on this input `z` and see what the results look like.

```
kmeans(z, centers=2)
```

K-means clustering with 2 clusters of sizes 30, 30

Cluster means:

	x	y
1	-2.913383	2.915673
2	2.915673	-2.913383

Clustering vector:

[illegible]

Within cluster sum of squares by cluster:

```
[1] 63.77404 63.77404
(between_SS / total_SS = 88.9 %)
```

Available components:

```
[1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
[6] "betweenss"    "size"         "iter"         "ifault"
```

```
km <- kmeans(z,centers=2)
attributes(km)
```

```
$names
[1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
[6] "betweenss"    "size"         "iter"         "ifault"

$class
[1] "kmeans"
```

Q. How many points are in each cluster?

```
km$size
```

```
[1] 30 30
```

Q. What “component of your results object details cluster assignment/ membership?

```
km$cluster
```

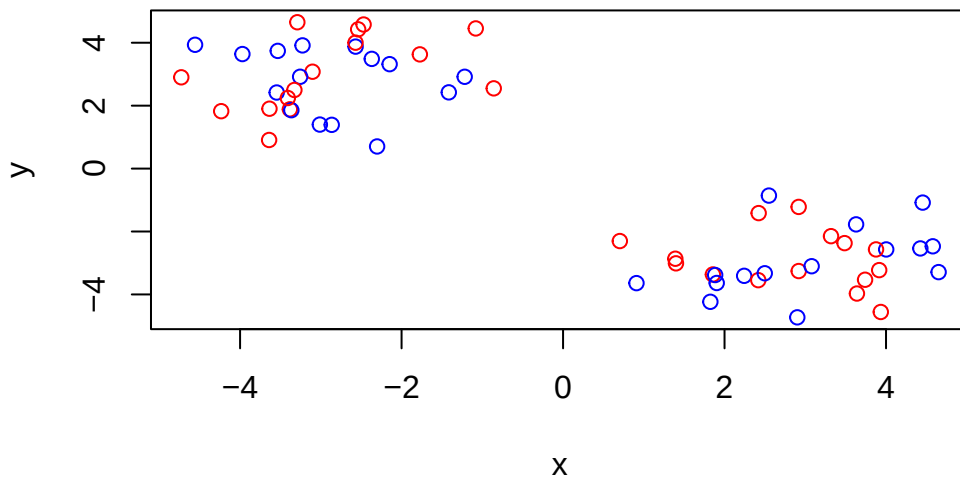
```
[1] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 1 1 1 1 1 1 1 1
[39] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
```

```
km$centers
```

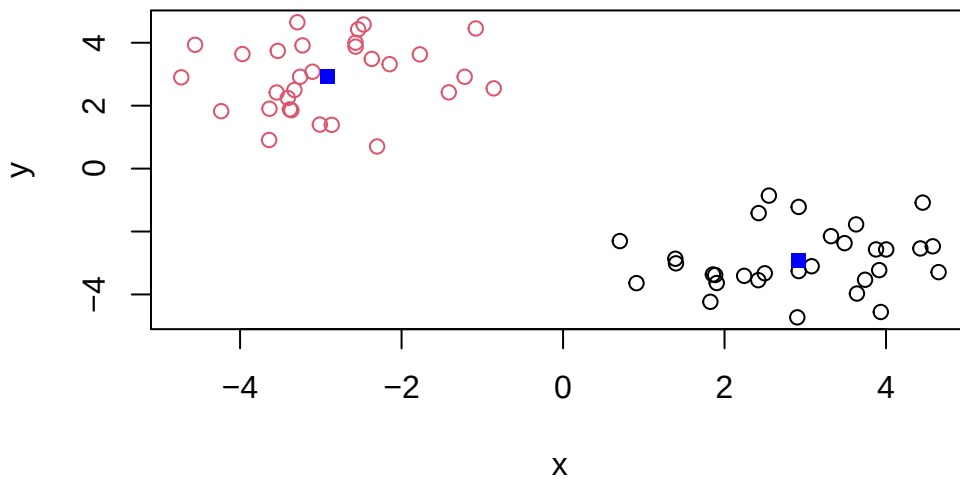
```
      x      y
1  2.915673 -2.913383
2 -2.913383  2.915673
```

Q. Plot z colored by the kmeans cluster assignment and add cluster centers as blue points

```
plot(z, col=c("red","blue"))
```

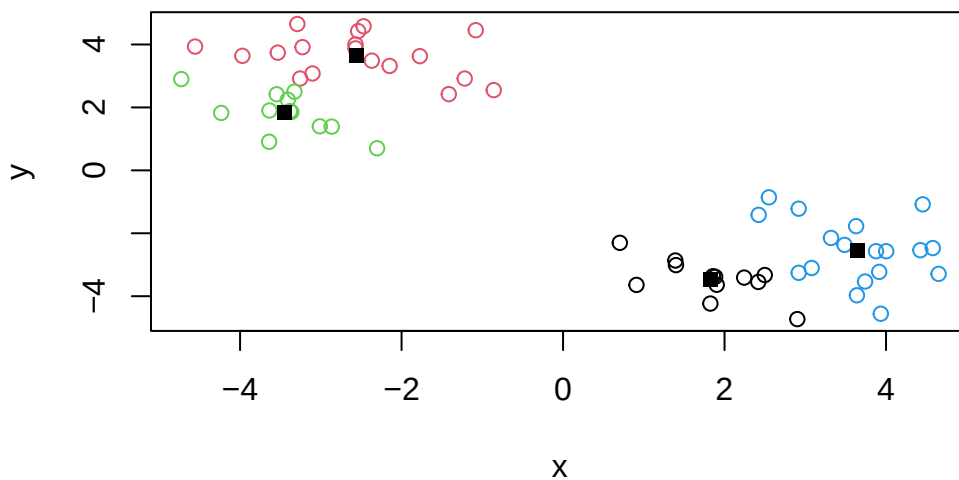


```
plot(z, col=km$cluster)
points(km$centers, col="blue", pch=15)
```



Q. Run a K-means clustering and plot the results asking for 4 clusters (K=4)?

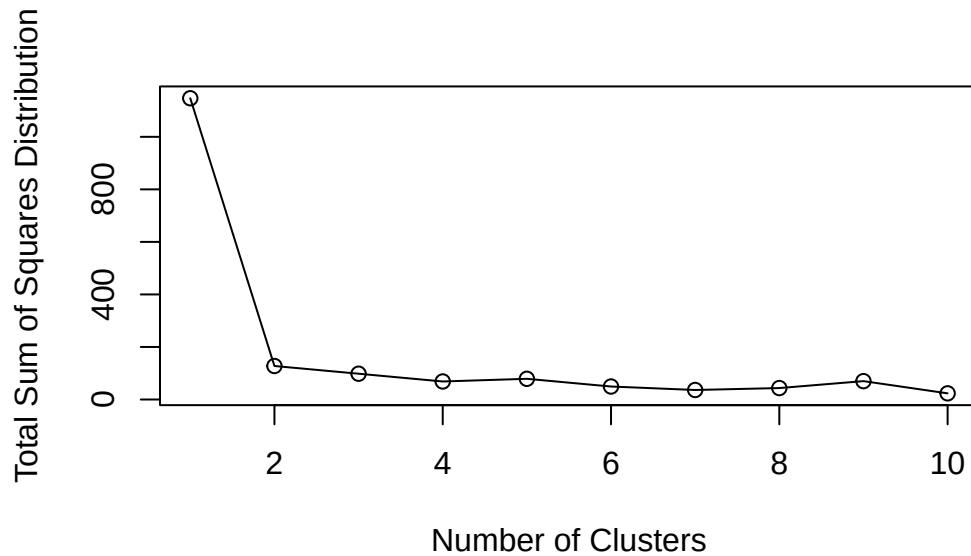
```
km4 <- kmeans(z, centers=4)
plot(z, km4$clusters, col=km4$cluster)
points(km4$centers, col="black", pch=15)
```



N.B. You need to tell K-means the number of clusters (i.e. set `centers=2`)

One approach is to try different values for `centers` and then pick the best...

```
ans <- NULL
for (i in 1:10) {
  km <- kmeans(z, centers=i)
  ans <- c(ans, km$tot.withinss)
}
plot (ans, typ="o", xlab="Number of Clusters", ylab="Total Sum of Squares Distribution")
```



Hierarchical Clustering

The main function in “base” R for Hierarchical Clustering is called `hclust()`

This function does not take your “raw” data for clustering. You must first build a “distance matrix” from your data and pass this as input to `hclust()`.

```
d <- dist(z)
hc <- hclust(d)
hc
```

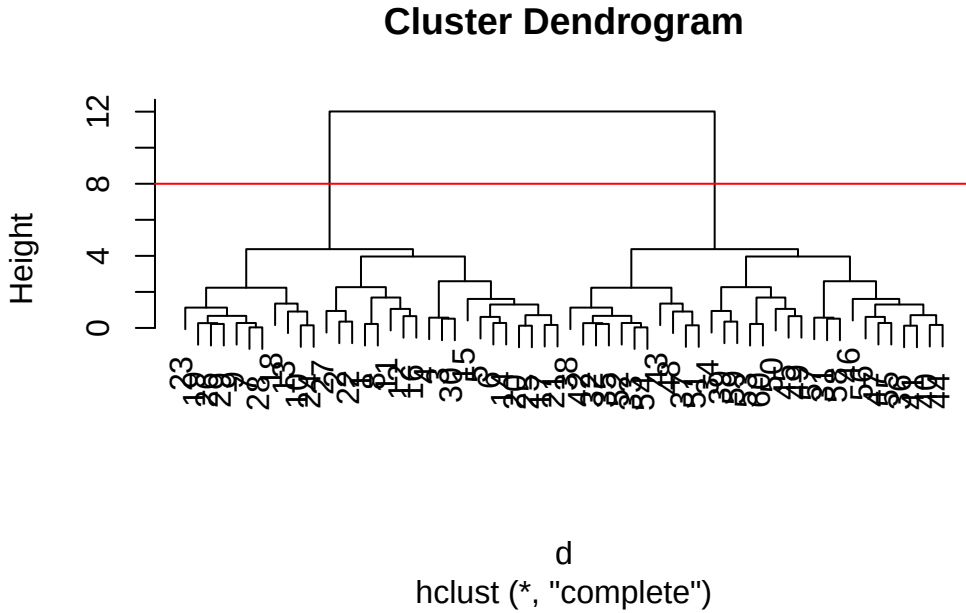
Call:

```
hclust(d = d)
```

```
Cluster method : complete
Distance       : euclidean
Number of objects: 60
```

There is a bespoke `plot()` method for `hclust()` result objects.

```
plot(hc)
abline(h=8, col="red")
```



Once we have our `hclust()` object (our “tree” of “cluster dendrogram”) we can “*cut*” the tree to reveal the clustering pattern.

```
cutree(hc, h=8)
```

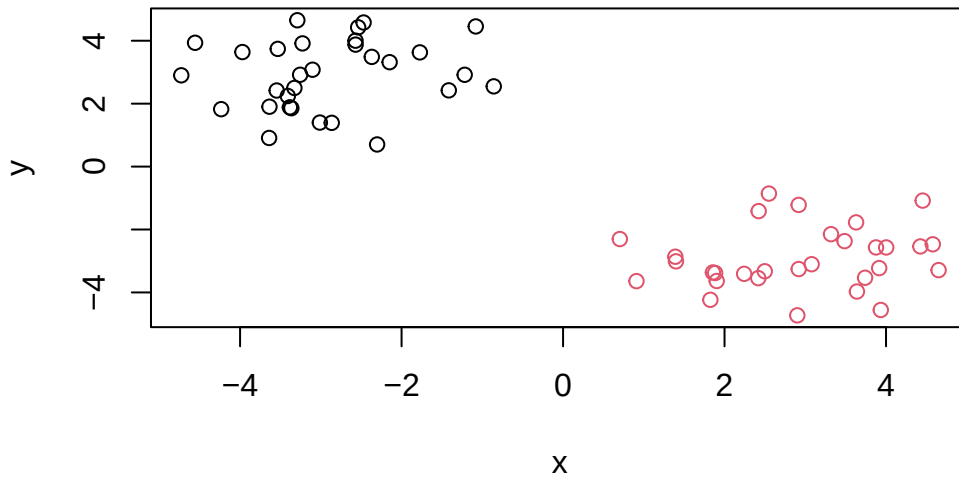
[illegible]

```
cutree(hc, k=4)
```

[1] 1 1 1 1 1 1 2 1 2 2 1 1 2 1 1 1 1 2 2 1 1 1 2 2 1 2 1 2 2 1 3 4 4 3 4 3 4 4
[39] 3 3 3 4 4 3 3 3 3 4 3 3 4 4 3 4 3 3 3 3 3 3

Q. Make a plot of $\mathbf{z}$ with your hclust results (i.e. colored by cluster membership)

```
grps <- cutree(hc, k=2)
plot(z, col=grps)
```



Principal Component Analysis (PCA)

PCA is a dimensionality reduction method that is popular for revealing patterns in complex datasets.

Analysis of UK food data

Let's look at some data on the eating habits of folks from the UK to see if there are patterns and trends that have some regions being distinct from others.

Data Import

The data is made available in CSV format so we can use the `read.csv` function for import to R:

```
url <- "https://tinyurl.com/UK-foods"
x <- read.csv(url)
x
```

	X	England	Wales	Scotland	N.Ireland
1	Cheese	105	103	103	66
2	Carcass_meat	245	227	242	267
3	Other_meat	685	803	750	586
4	Fish	147	160	122	93
5	Fats_and_oils	193	235	184	209
6	Sugars	156	175	147	139
7	Fresh_potatoes	720	874	566	1033
8	Fresh_Veg	253	265	171	143
9	Other_Veg	488	570	418	355
10	Processed_potatoes	198	203	220	187
11	Processed_Veg	360	365	337	334
12	Fresh_fruit	1102	1137	957	674
13	Cereals	1472	1582	1462	1494
14	Beverages	57	73	53	47
15	Soft_drinks	1374	1256	1572	1506
16	Alcoholic_drinks	375	475	458	135
17	Confectionery	54	64	62	41

Q1. How many rows and columns are in your new data frame named x? What R functions could you use to answer this questions?

The data frame x has 17 rows and 5 columns, and you can use functions like `dim(x)`, `nrow(x)`, and `ncol(x)` to determine this.

```
dim(x)
```

```
[1] 17  5
```

`nrow(x)` and `ncol(x)` can also be used to determine this.

Preview the first 6 rows

```
head(x)
```

```
head(x)
```

	X	England	Wales	Scotland	N.Ireland
1	Cheese	105	103	103	66
2	Carcass_meat	245	227	242	267
3	Other_meat	685	803	750	586

4	Fish	147	160	122	93
5	Fats_and_oils	193	235	184	209
6	Sugars	156	175	147	139

Note how the minus indexing works

```
rownames(x) <- x[,1]
x <- x[,-1]
head(x)
```

	England	Wales	Scotland	N.Ireland
Cheese	105	103	103	66
Carcass_meat	245	227	242	267
Other_meat	685	803	750	586
Fish	147	160	122	93
Fats_and_oils	193	235	184	209
Sugars	156	175	147	139

```
dim(x)
```

```
[1] 17  4
```

```
x <- read.csv(url, row.names=1)
head(x)
```

	England	Wales	Scotland	N.Ireland
Cheese	105	103	103	66
Carcass_meat	245	227	242	267
Other_meat	685	803	750	586
Fish	147	160	122	93
Fats_and_oils	193	235	184	209
Sugars	156	175	147	139

Tidy Data

Fix anything that went wrong with data import.

Exploratory Analysis

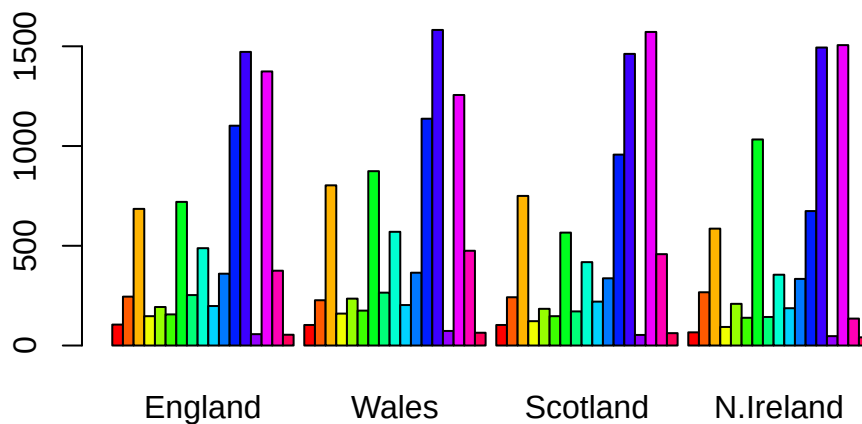
Q2. Which approach to solving the ‘row-names problem’ mentioned above do you prefer and why? Is one approach more robust than another under certain circumstances?

I prefer keeping row names as a regular column because it is clearer and easier to manipulate. It’s more robust because row names can be dropped or changed when converting data types or exporting data.

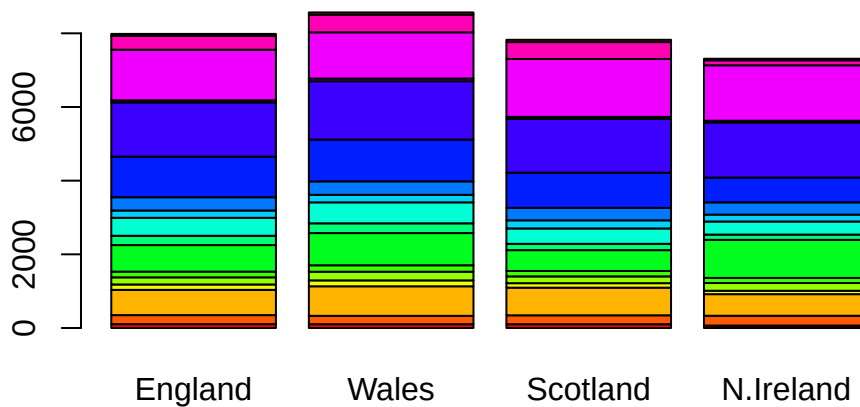
Make some plots to help make sense of obvious trends...

Barplots

```
barplot(as.matrix(x),  
        beside = TRUE,  
        col = rainbow(nrow(x)))
```



```
barplot(as.matrix(x),  
        beside = FALSE,  
        col = rainbow(nrow(x)))
```



Q3. Changing what optional argument in the above `barplot()` function results in the following plot?

Changing the optional argument beside from `TRUE` to `FALSE` makes the base R barplot stacked instead of grouped (see above).

```
library(tidyr)
library(ggplot2)
```

```
x_long <- x |>
  tibble::rownames_to_column("Food") |>
  pivot_longer(cols = -Food,
               names_to = "Country",
               values_to = "Consumption")
```

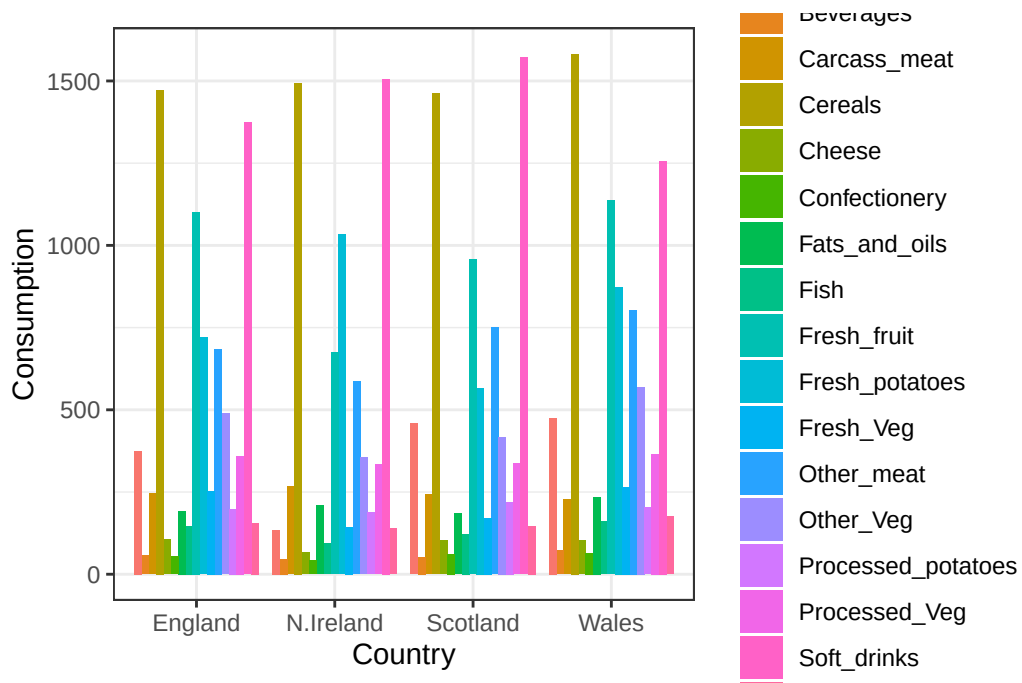
```
dim(x_long)
```

```
[1] 68  3
```

```
head(x_long)
```

```
# A tibble: 6 x 3
  Food      Country Consumption
  <chr>     <chr>      <int>
1 "Cheese"  England      105
2 "Cheese"  Wales        103
3 "Cheese"  Scotland     103
4 "Cheese"  N.Ireland     66
5 "Carcass_meat " England      245
6 "Carcass_meat " Wales        227
```

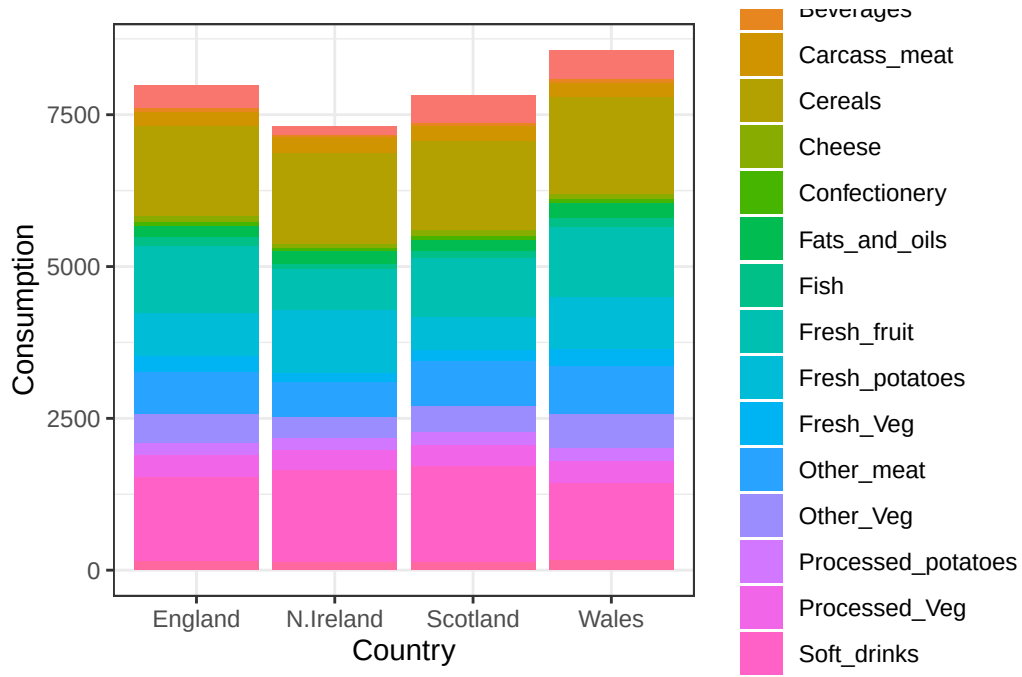
```
ggplot(x_long) +
  aes(x = Country, y = Consumption, fill = Food) +
  geom_col(position = "dodge") +
  theme_bw()
```



Q4.Changing what optional argument in the above ggplot() code results in a stacked barplot figure?

Changing the optional argument position in geom_col() from “dodge” to “stack” makes the ggplot figure a stacked barplot.

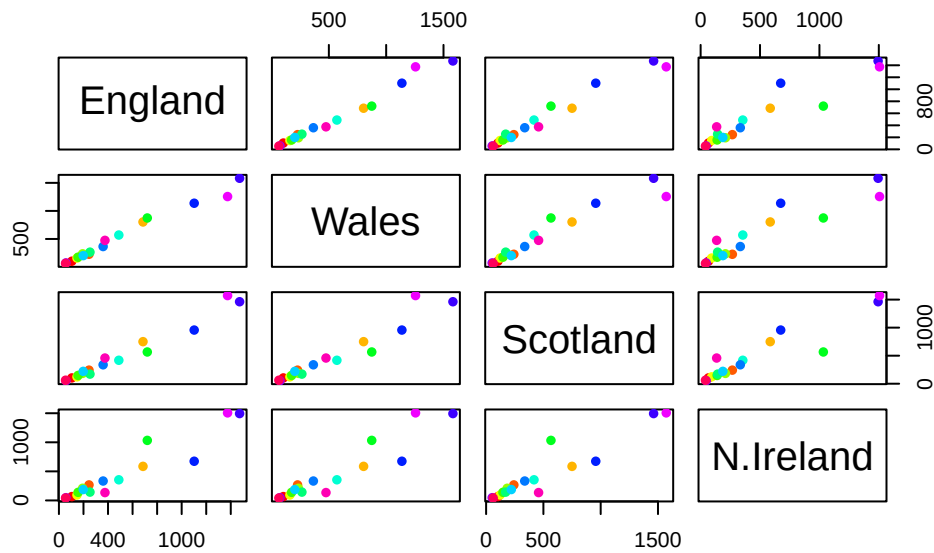
```
ggplot(x_long) +
  aes(x = Country, y = Consumption, fill = Food) +
  geom_col(position = "stack") +
  theme_bw()
```



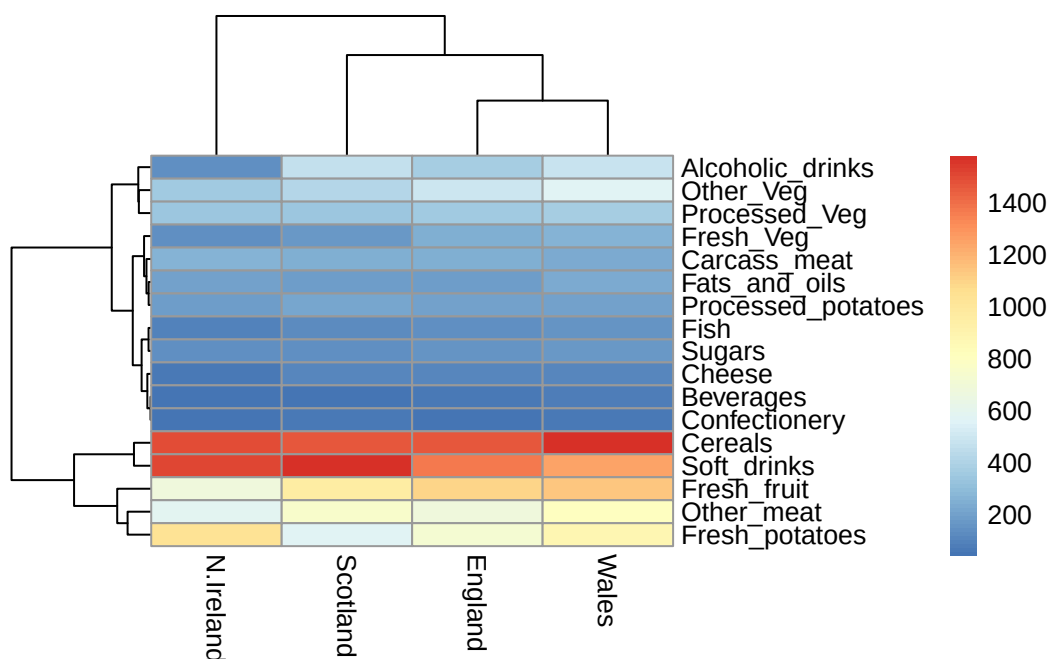
Q5. We can use the `pairs()` function to generate all pairwise plots for our countries. Can you make sense of the following code and resulting figure? What does it mean if a given point lies on the diagonal for a given plot?

A point on the diagonal means the two countries have equal values for that observation (same consumption for that food item).

```
pairs(x, col=rainbow(nrow(x)), pch=16)
```



```
library(pheatmap)
pheatmap(as.matrix(x))
```



Q6. Based on the pairs and heatmap figures, which countries cluster together and what does this suggest about their food consumption patterns? Can you easily tell what the main differences between N. Ireland and the other countries of the UK in terms of this data-set?

England, Scotland, and Wales cluster closely together, suggesting similar food consumption patterns, while Northern Ireland is more distinct, though the exact differences are not extremely obvious from the heatmap alone.

Key-point: Even relatively small datasets can prove challenging to interpret. This function wants the “observations” to be rows and the “variables” to be columns.

So here we need to take the transpose of our x input object.

```
pca <- prcomp(t(x))
summary(pca)
```

Importance of components:

	PC1	PC2	PC3	PC4
Standard deviation	324.1502	212.7478	73.87622	2.921e-14
Proportion of Variance	0.6744	0.2905	0.03503	0.000e+00
Cumulative Proportion	0.6744	0.9650	1.00000	1.000e+00

PCA to the rescue

The main function in “base” R for PCA is called `prcomp()`.

The returned `pca` object has components that we can use to make our main result figures:

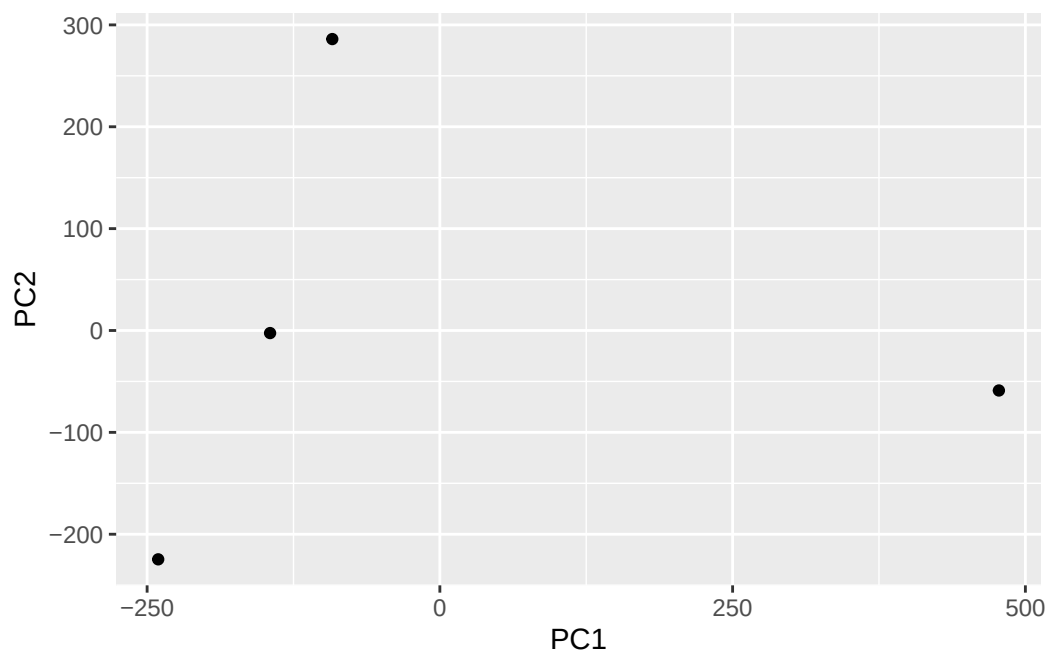
```
attributes(pca)
```

```
$names  
[1] "sdev"      "rotation" "center"    "scale"     "x"
```

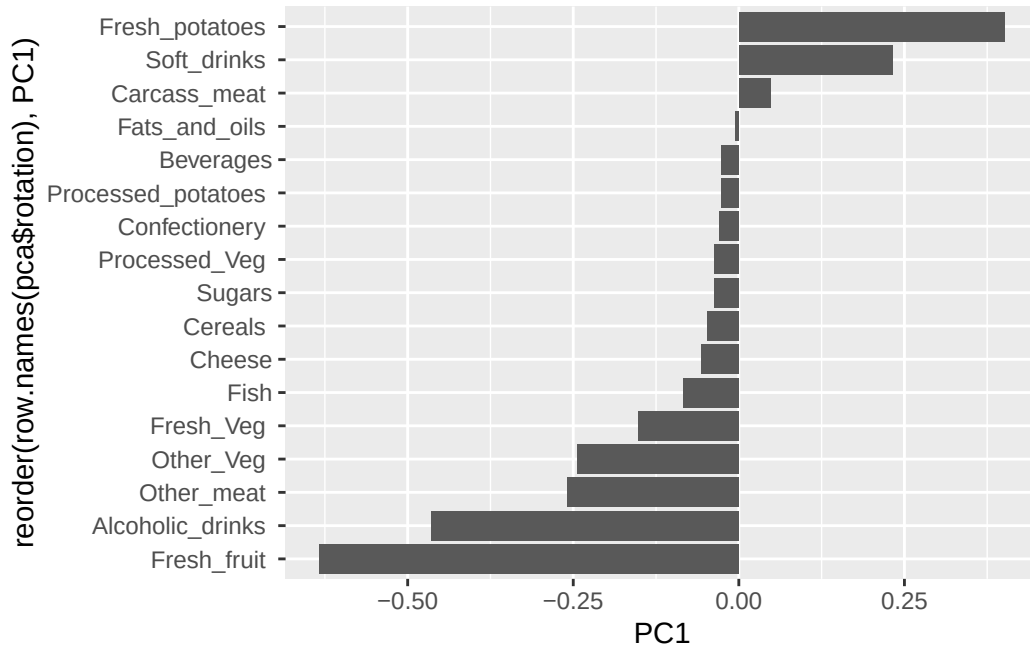
```
$class  
[1] "prcomp"
```

The main result figure from this analysis is called a “PC score plot” or “ordination plot” or “PC plot” or “PC1 vs PC2 plot”.

```
library(ggplot2)  
ggplot(pca$x) +  
  aes(PC1, PC2) +  
  geom_point()
```



```
ggplot(pca$rotation) +
  aes(PC1, reorder(row.names(pca$rotation), PC1)) +
  geom_col()
```



```
summary(pca)
```

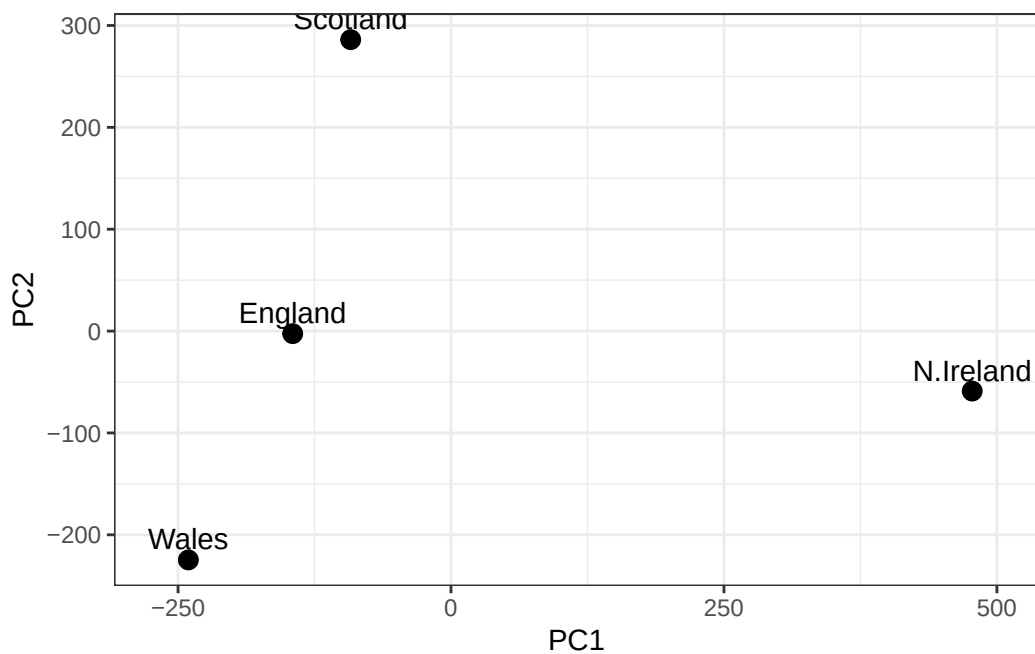
Importance of components:

	PC1	PC2	PC3	PC4
Standard deviation	324.1502	212.7478	73.87622	2.921e-14
Proportion of Variance	0.6744	0.2905	0.03503	0.000e+00
Cumulative Proportion	0.6744	0.9650	1.00000	1.000e+00

Q7. Complete the code below to generate a plot of PC1 vs PC2. The second line adds text labels over the data points.

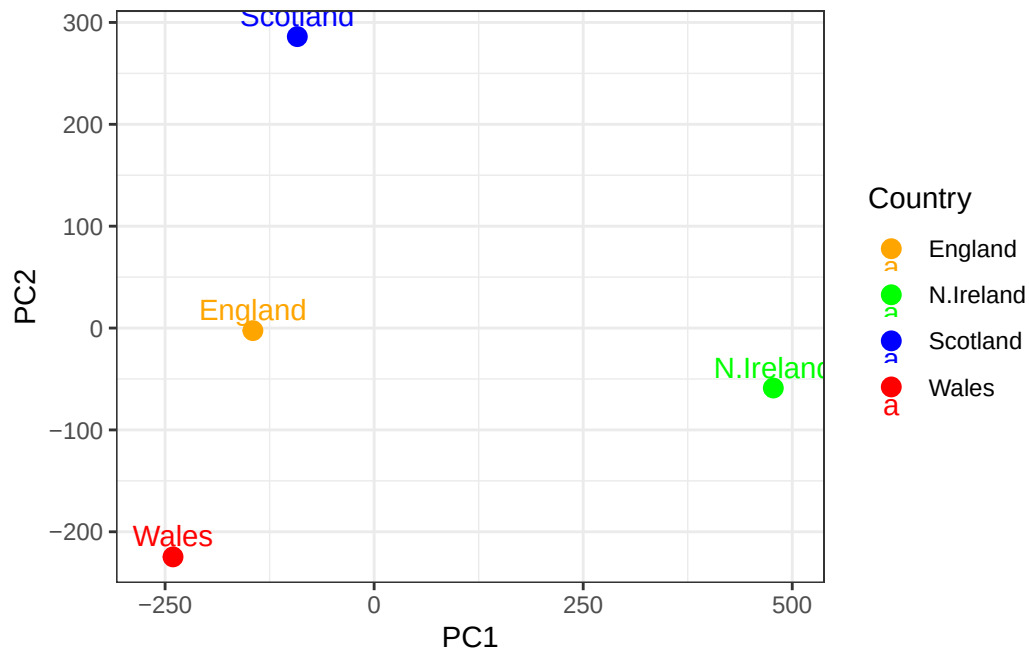
```
ggplot(pca$x) +
  aes(x = PC1, y = PC2, label = rownames(pca$x)) +
  geom_point(size = 3) +
  geom_text(vjust = -0.5) +
  xlim(-270, 500) +
  xlab("PC1") +
```

```
ylab("PC2") +  
theme_bw()
```



Q8. Customize your plot so that the colors of the country names match the colors in our UK and Ireland map and table at start of this document.

```
df <- as.data.frame(pca$x)  
df$Country <- rownames(df)  
ggplot(df) +  
  aes(x = PC1, y = PC2, label = Country, color = Country) +  
  geom_point(size = 3) +  
  geom_text(vjust = -0.5) +  
  xlim(-270, 500) +  
  xlab("PC1") +  
  ylab("PC2") +  
  scale_color_manual(values = c(  
    "England" = "orange",  
    "Scotland" = "blue",  
    "Wales" = "red",  
    "N.Ireland" = "green")) +  
  theme_bw()
```

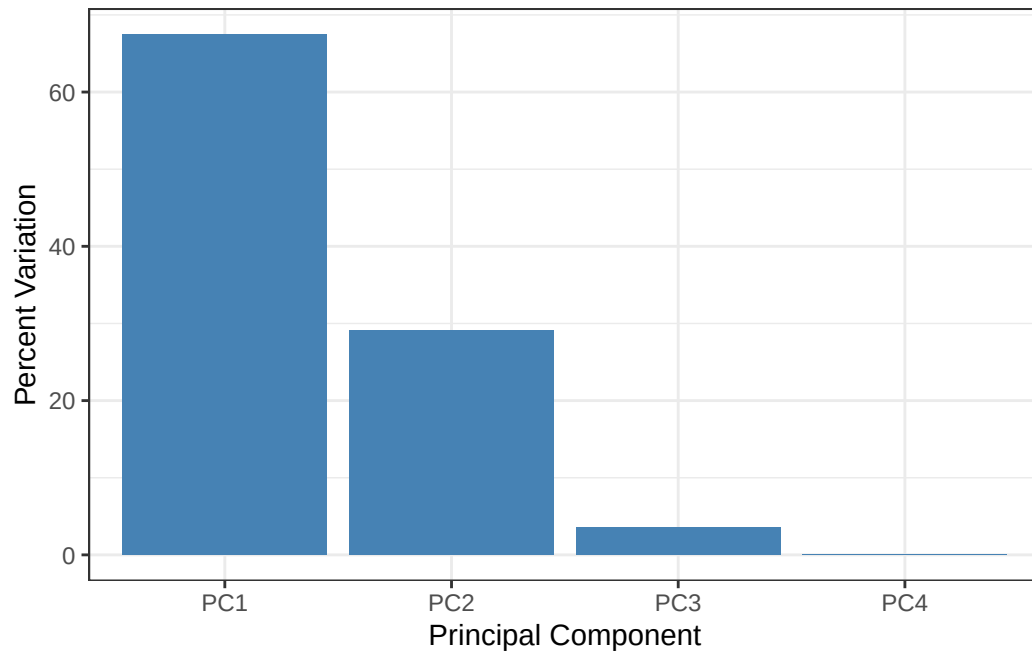


```
v <- (pca$sdev^2 / sum(pca$sdev^2)) * 100
```

```
v <- (pca$sdev^2 / sum(pca$sdev^2)) * 100
```

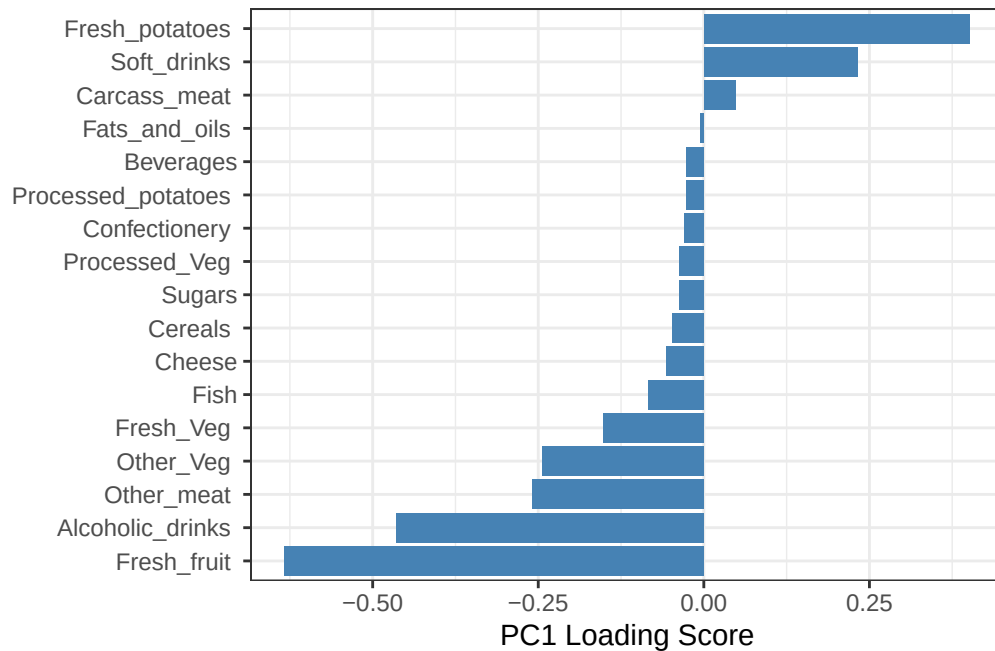
```
variance_df <- data.frame(
  PC = factor(paste0("PC", 1:length(v)), levels = paste0("PC", 1:length(v))),
  Variance = v
)
```

```
ggplot(variance_df) +
  aes(x = PC, y = Variance) +
  geom_col(fill = "steelblue") +
  xlab("Principal Component") +
  ylab("Percent Variation") +
  theme_bw() +
  theme(axis.text.x = element_text(angle = 0))
```



```
loadings_df <- as.data.frame(pca$rotation)
loadings_df$Feature <- rownames(loadings_df)

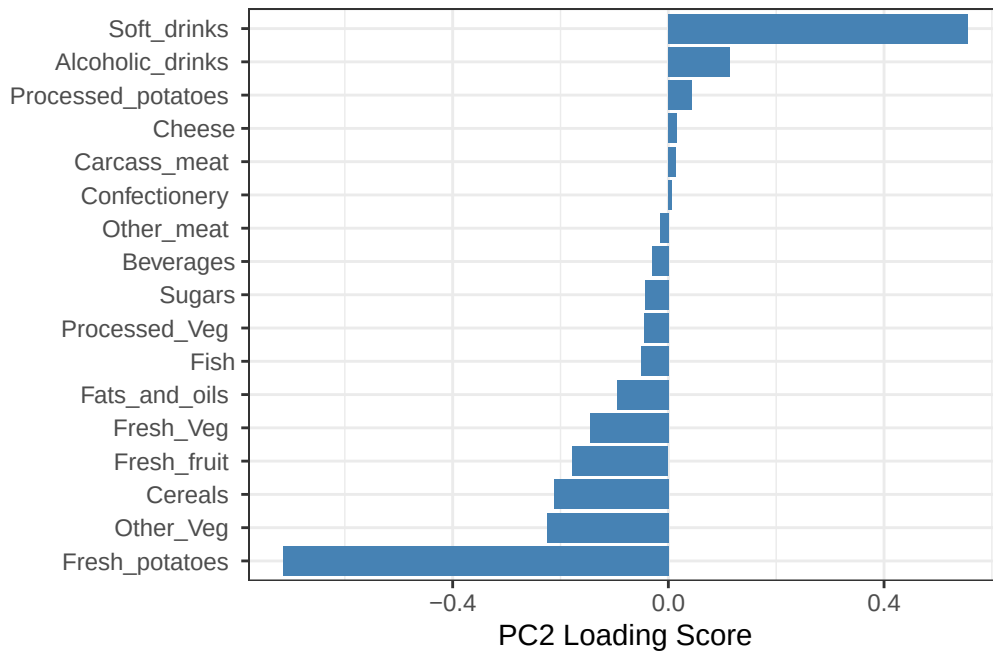
ggplot(loadings_df) +
  aes(x = PC1,
      y = reorder(Feature, PC1)) +
  geom_col(fill = "steelblue") +
  xlab("PC1 Loading Score") +
  ylab("") +
  theme_bw() +
  theme(axis.text.y = element_text(size = 9))
```



Q9: Generate a similar 'loadings plot' for PC2. What two food groups feature prominently and what does PC2 mainly tell us about?

PC2 is mainly driven by Fresh_fruit and Alcoholic_drinks, so it mostly reflects differences in fruit and alcohol consumption between countries (see below).

```
ggplot(loadings_df) +
  aes(x = PC2,
      y = reorder(Feature, PC2)) +
  geom_col(fill = "steelblue") +
  xlab("PC2 Loading Score") +
  ylab("") +
  theme_bw() +
  theme(axis.text.y = element_text(size = 9))
```



PCA of RNA-seq data

Q10: How many genes and samples are in this data set? How many PCs do you think it will take to have a useful overview of this data set (see below)?

There are 6 genes (rows) and 9 samples (columns), and it will likely take 2 principal components to provide a useful overview of the data.

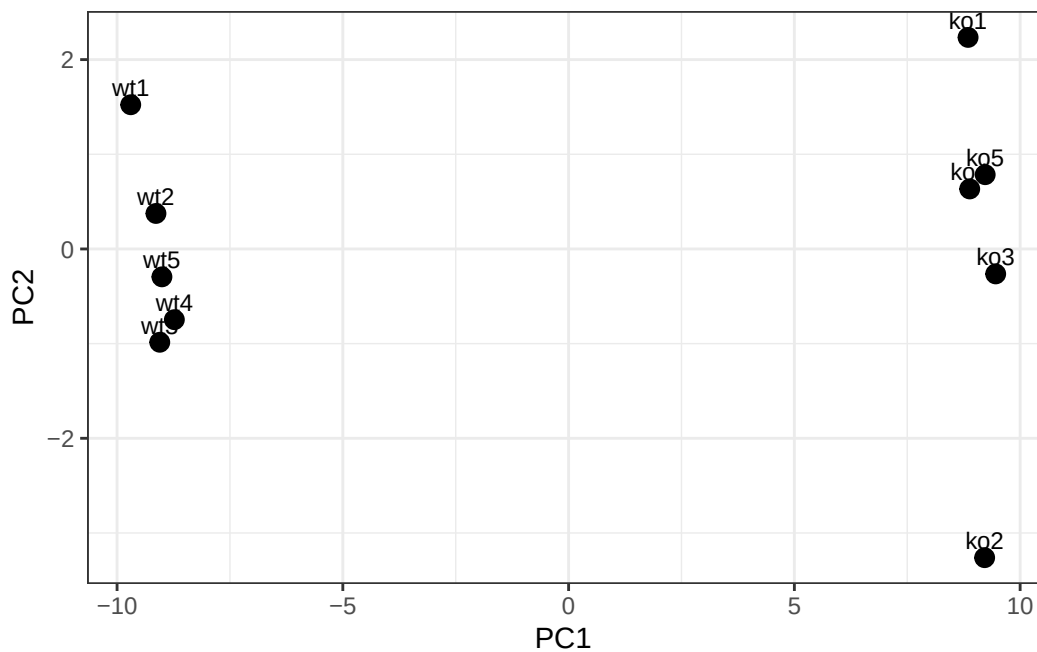
```
url2 <- "https://tinyurl.com/expression-CSV"
rna.data <- read.csv(url2, row.names=1)
head(rna.data)
```

	wt1	wt2	wt3	wt4	wt5	ko1	ko2	ko3	ko4	ko5
gene1	439	458	408	429	420	90	88	86	90	93
gene2	219	200	204	210	187	427	423	434	433	426
gene3	1006	989	1030	1017	973	252	237	238	226	210
gene4	783	792	829	856	760	849	856	835	885	894
gene5	181	249	204	244	225	277	305	272	270	279
gene6	460	502	491	491	493	612	594	577	618	638

```
## Again we have to take the transpose of our data
pca <- prcomp(t(rna.data), scale=TRUE)

# Create data frame for plotting
df <- as.data.frame(pca$x)
df$Sample <- rownames(df)

## Plot with ggplot
ggplot(df) +
  aes(x = PC1, y = PC2, label = Sample) +
  geom_point(size = 3) +
  geom_text(vjust = -0.5, size = 3) +
  xlab("PC1") +
  ylab("PC2") +
  theme_bw()
```



```
summary(pca)
```

Importance of components:

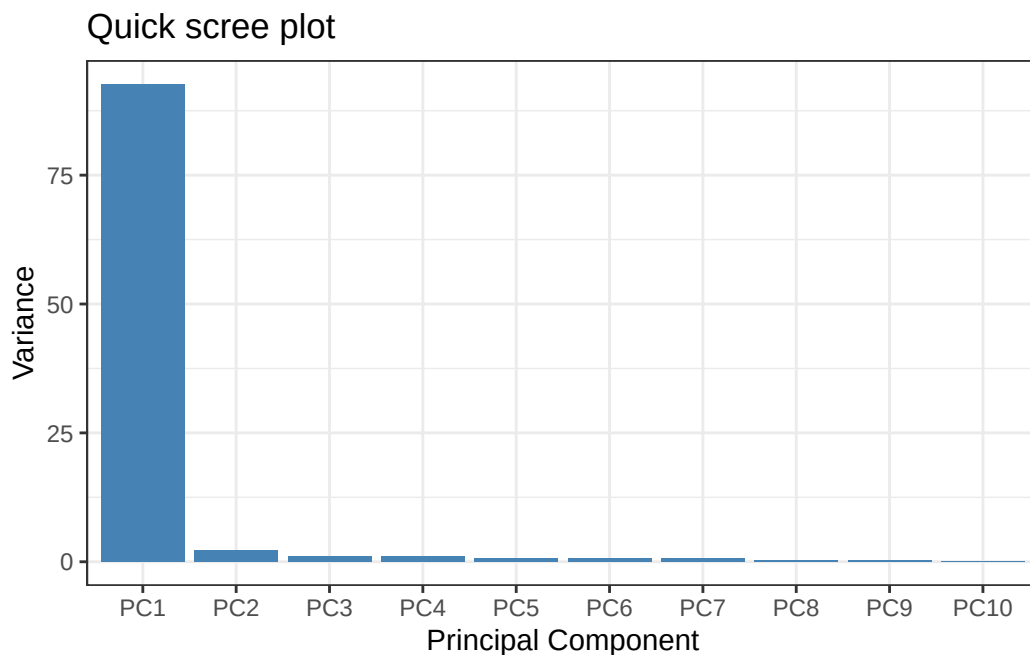
	PC1	PC2	PC3	PC4	PC5	PC6	PC7
Standard deviation	9.6237	1.5198	1.05787	1.05203	0.88062	0.82545	0.80111
Proportion of Variance	0.9262	0.0231	0.01119	0.01107	0.00775	0.00681	0.00642

Cumulative Proportion	0.9262	0.9493	0.96045	0.97152	0.97928	0.98609	0.99251
	PC8	PC9	PC10				
Standard deviation	0.62065	0.60342	3.3e-15				
Proportion of Variance	0.00385	0.00364	0.0e+00				
Cumulative Proportion	0.99636	1.00000	1.0e+00				

```
# Calculate variance explained
pca.var <- pca$sdev^2
pca.var.per <- round(pca.var/sum(pca.var)*100, 1)

# Create scree plot data
scree_df <- data.frame(
  PC = factor(paste0("PC", 1:10), levels = paste0("PC", 1:10)),
  Variance = pca.var[1:10]
)

ggplot(scree_df) +
  aes(x = PC, y = Variance) +
  geom_col(fill = "steelblue") +
  ggtitle("Quick scree plot") +
  xlab("Principal Component") +
  ylab("Variance") +
  theme_bw()
```

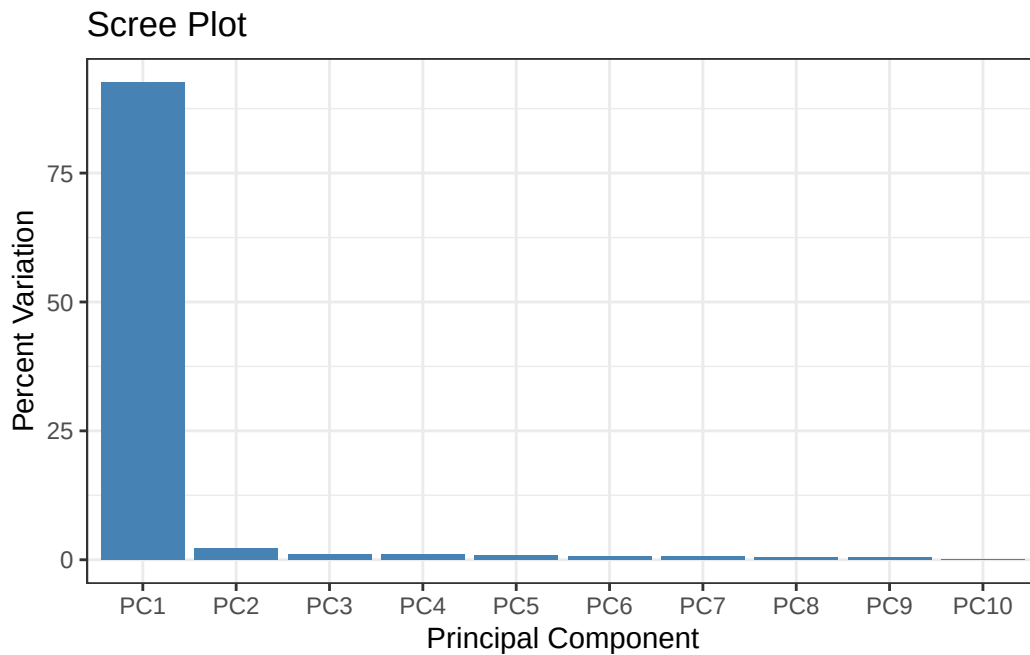


```
pca.var.per
```

```
[1] 92.6  2.3  1.1  1.1  0.8  0.7  0.6  0.4  0.4  0.0
```

```
# Create percent variance scree plot
scree_pct_df <- data.frame(
  PC = factor(paste0("PC", 1:10), levels = paste0("PC", 1:10)),
  PercentVariation = pca.var.per[1:10]
)

ggplot(scree_pct_df) +
  aes(x = PC, y = PercentVariation) +
  geom_col(fill = "steelblue") +
  ggtitle("Scree Plot") +
  xlab("Principal Component") +
  ylab("Percent Variation") +
  theme_bw()
```

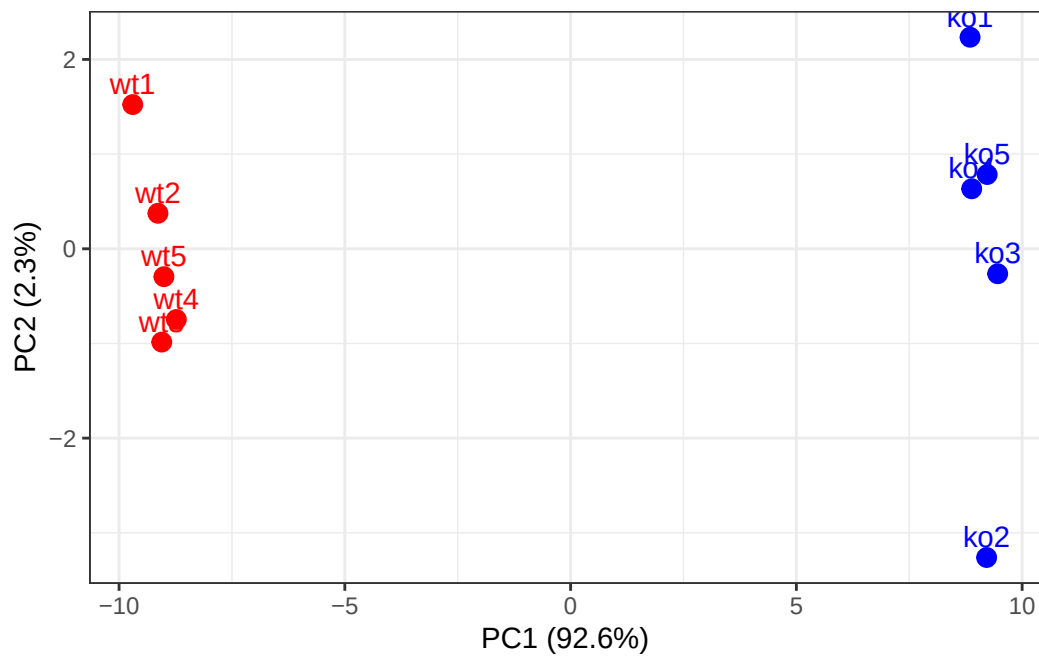


```
## A vector of colors for wt and ko samples
colvec <- colnames(rna.data)
colvec[grep("wt", colvec)] <- "red"
```

```
colvec[grep("ko", colvec)] <- "blue"

# Add condition to data frame
df$condition <- substr(df$Sample, 1, 2)
df$color <- colvec

ggplot(df) +
  aes(x = PC1, y = PC2, color = color, label = Sample) +
  geom_point(size = 3) +
  geom_text(vjust = -0.5, hjust = 0.5, show.legend = FALSE) +
  scale_color_identity() +
  xlab(paste0("PC1 (", pca.var.per[1], "%)")) +
  ylab(paste0("PC2 (", pca.var.per[2], "%)")) +
  theme_bw()
```



```
loading_scores <- pca$rotation[,1]

## Find the top 10 measurements (genes) that contribute
## most to PC1 in either direction (+ or -)
gene_scores <- abs(loading_scores)
gene_score_ranked <- sort(gene_scores, decreasing=TRUE)
```

```
## show the names of the top 10 genes  
top_10_genes <- names(gene_score_ranked[1:10])  
top_10_genes
```

```
[1] "gene100" "gene66"  "gene45"  "gene68"  "gene98"  "gene60"  "gene21"  
[8] "gene56"  "gene10"  "gene90"
```